

Machine learning (ML):

- Machine learning (ML) is a subset of artificial intelligence (AI) that enables computers to learn from data and improve their performance over time without being explicitly programmed.
- Instead of following strict, hard-coded rules, ML models identify patterns in data to make predictions, recommendations, or generate new content.

Key Aspects of Machine Learning:

- Data-Driven: ML algorithms require large amounts of data to find patterns and build models.
- Prediction and Decision Making: Models are trained to forecast outcomes (e.g., house prices) or classify information (e.g., identifying spam emails).
- Automation and Improvement: As these systems are exposed to more data, their accuracy improves over time.

Types of ML:

- Supervised Learning: Training on labeled data (input and output are provided).
- Unsupervised Learning: Finding hidden patterns in unlabeled data.
- Reinforcement Learning: Learning through trial and error to achieve a goal.

Supervised learning

- Supervised learning is a category of machine learning where a computer algorithm is trained using a labeled dataset. In this paradigm, each piece of input data (known as "features") is paired with its corresponding correct output (known as a "label" or "target").

Core Categories

Supervised learning tasks generally fall into two main types based on the nature of the output:

1. Classification: Used when the output is a categorical or discrete value. The goal is to sort inputs into predefined classes.
- Examples: Identifying an email as "Spam" or "Not Spam"; distinguishing between images of cats and dogs.

2. Regression: Used when the output is a continuous or numerical value. The goal is to understand the relationship between variables to predict a specific quantity.
- Examples: Predicting house prices based on square footage; forecasting future stock prices or temperatures.

Linear Regression:

Linear Regression is a fundamental supervised learning algorithm used to model the relationship between a dependent variable and one or more independent variables. It predicts continuous values by fitting a straight line that best represents the data.

- It assumes that there is a linear relationship between the input and output
- Uses a best-fit line to make predictions
- Commonly used in forecasting, trend analysis, and predictive modelling
- For simple linear regression (with one independent variable), the best-fit line is represented by the equation: **$y=mx+b$**

Where:

- y is the predicted value (dependent variable)
- x is the input (independent variable)
- m is the slope of the line (how much y changes when x changes)
- b is the intercept (the value of y when x = 0)

Example

Data: (X, Y) (1, 2) (2, 4) (3, 5) (4, 4) (5, 6)

- $n = 5$
- $\sum x = 15$
- $\sum y = 21$
- $\sum x^2 = 55$
- $\sum xy = 71$

1. Calculate Summations:

2. Calculate Slope (b):

$$b = \frac{n(\sum xy) - (\sum x)(\sum y)}{n(\sum x^2) - (\sum x)^2} = \frac{5(71) - (15)(21)}{5(55) - (15)^2} = \frac{355 - 315}{275 - 225} = \frac{40}{50} = 0.8$$

3. Calculate Intercept (a):

$$a = \frac{\sum y - b(\sum x)}{n} = \frac{21 - 0.8(15)}{5} = \frac{21 - 12}{5} = \frac{9}{5} = 1.8$$

4. Regression Line:

$$y = 1.8 + 0.8x$$

5. **Prediction:** If 6g of fertilizer is used ($x = 6$), predicted yield $y = 1.8 + 0.8(6) = 6.6$ kg.

```
In [2]: import numpy as np
import pandas as pd
data={'X':[1, 2, 3, 4, 5],
      'Y':[2, 4, 5, 4, 6]}
df=pd.DataFrame(data)
df
```

```
Out[2]:
```

	X	Y
0	1	2
1	2	4
2	3	5
3	4	4
4	5	6

```
In [3]: df.corr(numeric_only=True)
```

```
Out[3]:
```

	X	Y
X	1.000000	0.852803
Y	0.852803	1.000000

```
In [7]: from sklearn.linear_model import LinearRegression

X = df[['X']]
y = df[['Y']]

model = LinearRegression()
model.fit(X, y)
```

```
Out[7]:
```

LinearRegression ⓘ ⓘ
LinearRegression()

```
In [15]: y_pred = model.predict([[6]])
print(y_pred)
```

```
[6.6]
```

C:\Users\shivangi\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names

```
warnings.warn(
```

coefficient

A coefficient in linear regression is a numerical value that represents the relationship between an independent variable (predictor) and a dependent variable (outcome). It indicates the expected change in the dependent variable for a one-unit increase in the independent variable, holding other variables constant.

```
In [11]: print(model.coef_)
```

```
[0.8]
```

intercept

The intercept in linear regression is the expected mean value of the dependent variable (Y) when all independent variables (X) are zero.

```
In [13]: print(model.intercept_)
```

```
1.7999999999999994
```

Simple Linear Regression on Fish.csv

```
In [ ]: import numpy as np
import pandas as pd

df=pd.read_csv("Fish.csv")

print(df.corr(numeric_only=True))
```

```
In [ ]: from sklearn.model_selection import train_test_split
X = df[['Length3']]
y = df['Weight']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_stat
```

```
In [ ]: from sklearn.linear_model import LinearRegression
# Fit the model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
In [ ]: y_pred = model.predict(X_test)
```

R-Squared (R^2)

R-squared, also known as the coefficient of determination, measures the proportion of the variance in the dependent variable that is predictable from the independent variables. In simpler terms, it tells us how well the regression tree model fits the data.

Formula

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Where:

$$SS_{res} \text{ (Residual Sum of Squares)} = \sum (y_i - \hat{y}_i)^2$$

$$SS_{tot} \text{ (Total Sum of Squares)} = \sum (y_i - \bar{y})^2$$

y_i = Actual value

$\hat{y}_i$ = Predicted value

$\bar{y}$ = Mean of the actual values

Interpretation

- $R^2 = 1$: Perfect fit. The model explains all the variability of the response data around its mean.
- $R^2 = 0$: The model does not explain any variability of the response data around its mean.
- $0 < R^2 < 1$: The model explains a portion of the variability.
- $R^2 < 0$: The model is worse than a horizontal line (mean of the target variable).

Mean Squared Error (MSE)

Mean Squared Error measures the average of the squares of the errors — that is, the average squared difference between the actual and predicted values. It gives an idea of the magnitude of the errors in your model's predictions.

Formula

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where:

n = Number of data points

y_i = Actual value

$\hat{y}_i$ = Predicted value

Interpretation

- **MSE = 0:** Perfect predictions. There are no errors in the model's predictions.
- **Higher MSE:** Indicates larger average squared differences between the actual and predicted values, signifying poor model performance.

Using R² and MSE Together

While R-squared provides a measure of how well the model explains the variability of the target variable, MSE gives insight into the magnitude of prediction errors. Together, they provide a comprehensive view of model performance.

Mean Absolute Error (MAE)

Mean Absolute Error (MAE) is a regression metric that measures the average magnitude of errors between predicted and actual values by calculating the absolute difference between them. It is used to evaluate model performance by providing an average error score in the same units as the target variable. MAE is often called L1 loss.

Formula:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

- Robust to Outliers: Unlike Mean Squared Error (MSE), MAE does not severely penalize large outliers, making it useful for noisy data.
- Linearity: It treats all errors equally; a 10-point error is twice as bad as a 5-point error.
- Interpretation: A lower MAE (closer to 0) indicates better performance.

```
In [1]: import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

data={'X':[1, 2, 3, 4, 5],
      'Y':[2, 4, 5, 4, 6]}
df=pd.DataFrame(data)

X = df[['X']]
y = df['Y']

model = LinearRegression()
model.fit(X, y)
y_pred = model.predict([[1],[2],[3],[4],[5]])
print(y_pred)
```

```
[2.6 3.4 4.2 5.  5.8]
```

```
C:\Users\shivangi\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2739: User
Warning: X does not have valid feature names, but LinearRegression was fitted with f
eature names
warnings.warn(
```

r2_score

```
In [2]: from sklearn.metrics import r2_score
r2 = r2_score(y, y_pred)
print(f'R2 Score: {r2}')
```

```
R2 Score: 0.7272727272727274
```

Mean Squared Error (MSE)

```
In [3]: from sklearn.metrics import mean_squared_error
mse= mean_squared_error(y, y_pred)
print(f'Mean Squared Error: {mse}')
```

```
Mean Squared Error: 0.47999999999999998
```

Mean Absolute Error (MAE)

```
In [4]: from sklearn.metrics import mean_absolute_error
mae= mean_absolute_error(y, y_pred)
print(f'Mean Squared Error: {mae}')
```

Mean Squared Error: 0.6399999999999998

Multiple Linear Regression

Multiple linear regression (MLR) is a statistical technique that models the linear relationship between one continuous dependent variable (target) and two or more independent variables (predictors).

It predicts outcomes by fitting a linear equation,

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n$$

```
In [7]: import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

fish_data=pd.read_csv("Fish.csv")

X = fish_data[['Length1', 'Length2', 'Length3', 'Height', 'Width']] # Multiple Inde
y = fish_data['Weight']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_stat

# Fit the model
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Calculate R2 score
r2 = r2_score(y_test, y_pred)
print(f'R2 Score: {r2}')
```

R2 Score: 0.9400086678556974

Mean Squared Error: 8507.327045832022

Polynomial Regression

- Polynomial regression is a form of regression analysis that models the relationship between an independent variable X and a dependent variable Y as an n-th degree

polynomial, allowing it to fit non-linear, curved data.

Key Aspects of Polynomial Regression:

- **Non-linear Relationships:** While polynomial regression models data that is not straight (non-linear), it is still considered a special case of multiple linear regression because it is linear in terms of the coefficients ($w_0, w_1, w_2, \dots$).

- **Formula:** The general equation for a polynomial regression model of degree n is:

$$y = w_0 + w_1x + w_2x^2 + w_3x^3 + \dots + w_nx^n + \epsilon$$

- **When to Use:** Use polynomial regression when the relationship between variables shows curvature rather than a straight line. Examples include studying the growth rate of tissues, the distribution of carbon isotopes in lake sediments, or non-linear salary trends based on experience.
- **Overfitting Risk:** While higher-degree polynomials can better fit training data, they often lead to overfitting, where the model performs poorly on unseen, new data.
- **Advantages:**
 - It is highly flexible and can model a broad range of curvatures.
 - It offers a better fit for complex datasets.
- **Disadvantages:**
 - The model can become unstable if high-degree polynomials are used.

```
In [13]: import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

fish_data=pd.read_csv("Fish.csv")

X = fish_data[['Length1', 'Length2', 'Length3', 'Height', 'Width']]
y = fish_data['Weight']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_stat

# Transform features to polynomial features
poly = PolynomialFeatures(degree=3)
X_poly_train= poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

# Fit the model
model = LinearRegression()
model.fit(X_poly_train, y_train)
y_pred = model.predict(X_poly_test)
```

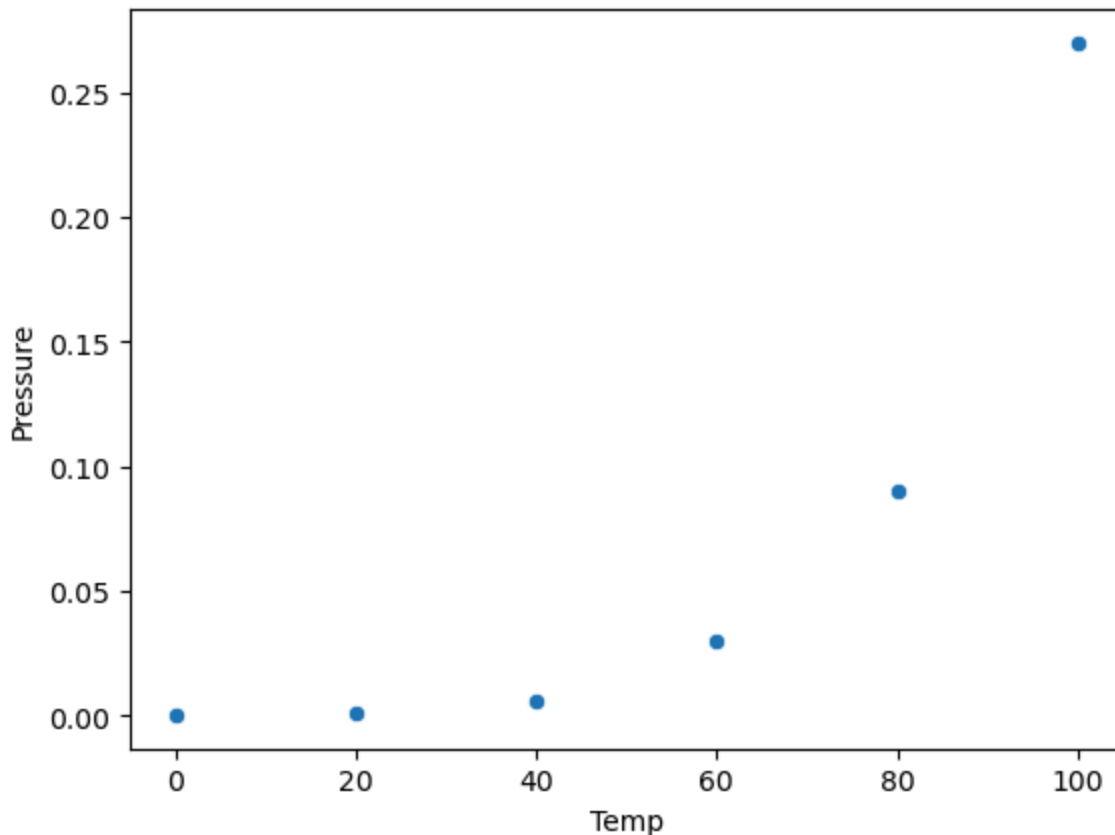
```
# Calculate R2 score
r2 = r2_score(y_test, y_pred)
print(f'R2 Score: {r2}')
```

R2 Score: 0.9743766706964123

Example of a type of data that has a non-linear relationship and is suitable for Polynomial Regression.

```
In [16]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data={'Temp':[0, 20, 40, 60, 80, 100],
      'Pressure':[0.0002, 0.0012, 0.0060, 0.0300, 0.0900, 0.2700]}
df=pd.DataFrame(data)
sns.scatterplot(x="Temp", y="Pressure", data=df)
plt.show()
```



```
In [21]: import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
```

```

data={'Temp':[0, 20, 40, 60, 80, 100],
      'Pressure':[0.0002, 0.0012, 0.0060, 0.0300, 0.0900, 0.2700]}
df=pd.DataFrame(data)

X = df[['Temp']]
y = df['Pressure']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_stat

# Fit the model
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

# Calculate R2 score
r2 = r2_score(y_test, y_pred)
print(f'R2 Score: {r2}')

```

R2 Score: 0.20034223522242245

```

In [22]: import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

data={'Temp':[0, 20, 40, 60, 80, 100],
      'Pressure':[0.0002, 0.0012, 0.0060, 0.0300, 0.0900, 0.2700]}
df=pd.DataFrame(data)

X = df[['Temp']]
y = df['Pressure']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_stat

# Transform features to polynomial features
degree = 3 # Example degree
poly = PolynomialFeatures(degree=degree)
X_poly_train= poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

# Fit the model
model = LinearRegression()
model.fit(X_poly_train, y_train)

y_pred = model.predict(X_poly_test)

# Calculate R2 score
r2 = r2_score(y_test, y_pred)
print(f'R2 Score: {r2}')

```

R2 Score: 0.8758710430424944

Polynomial Regression on Ice_cream selling data

The code below compares the performance of Linear Regression and Polynomial Regression on the same CSV file using R^2 score and Mean Squared Error. The Ice Cream Selling dataset has a non-linear relationship, which can be clearly observed in the scatter plot.

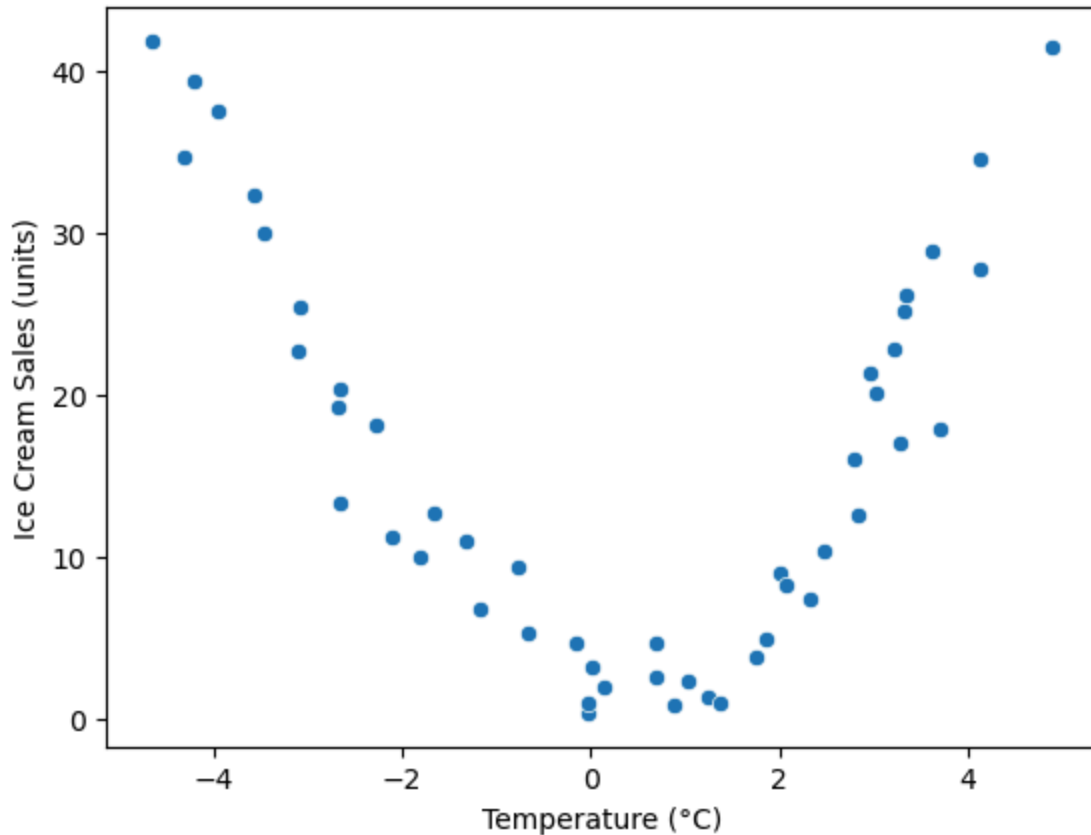
```
In [17]: df=pd.read_csv("Ice_cream selling data.csv")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 49 entries, 0 to 48
Data columns (total 2 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Temperature (°C)      49 non-null    float64
1   Ice Cream Sales (units) 49 non-null    float64
dtypes: float64(2)
memory usage: 916.0 bytes
```

```
In [18]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df=pd.read_csv("Ice_cream selling data.csv")

sns.scatterplot(x="Temperature (°C)", y="Ice Cream Sales (units)", data=df)
plt.show()
```



```
In [9]: import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

df=pd.read_csv("Ice_cream selling data.csv")

X = df[['Temperature (°C)']]
y = df['Ice Cream Sales (units)']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Fit the model
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

# Calculate R2 score
r2 = r2_score(y_test, y_pred) # Closer to 1 is good, closer to 0 is poor.
print(f'R2 Score: {r2}')

# Calculate MSE
mse= mean_squared_error(y_test, y_pred) # MSE ,less is better
print(f'Mean Squared Error: {mse}')
```

R2 Score: -0.3890788577957518

Mean Squared Error: 94.22738223229766

```
In [10]: import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

df=pd.read_csv("Ice_cream selling data.csv")

X = df[['Temperature (°C)']]
y = df['Ice Cream Sales (units)']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Transform features to polynomial features
degree = 3 # Example degree
poly = PolynomialFeatures(degree=degree)
X_poly_train= poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

# Fit the model
model = LinearRegression()
model.fit(X_poly_train, y_train)

y_pred = model.predict(X_poly_test)

# Calculate R2 score
r2 = r2_score(y_test, y_pred)
print(f'R2 Score: {r2}')

# Calculate MSE
mse= mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
```

R2 Score: 0.8038545160525787

Mean Squared Error: 13.30541846874058

In []: